Assignment 2 - Secure Copy And Authenticated IRC Chat

This directory implements both parts of Assignment 2.

- Section 1.1 secure file copy is implemented by `secure_client` and `secure_server`.
- Section 1.2 multi-user IRC/KDC chat is implemented by `client_real` and `server_real`.

Shared OpenSSL/socket helpers are in `a2_common.c` and `a2_common.h`.

Build

```sh
make clean && make
```

The Makefile builds:

- `secure_client`
- `secure_server`
- `client_real`
- `server_real`

OpenSSL Dependency

The build uses OpenSSL headers and libraries from `Assignment2/.deps/openssl` when present. To prepare that prefix:

```sh
make deps
```

`make deps` downloads OpenSSL 3.5.7 from the official OpenSSL GitHub release, checks SHA256, and then attempts a local source build. If the local Perl installation cannot run OpenSSL `Configure` because `IPC::Cmd` is missing, the target copies the existing workspace OpenSSL cache into `.deps/openssl` after source verification. The acquisition note is in `SAFETY.md`.

Demo Material

Create a secure-copy shared key:

```sh
make demo-key
```

Create a self-signed TLS certificate for file-request receiver mode:

```sh
make certs
```

Generated keys, certs, binaries, object files, and `.deps/` are ignored by `.gitignore`.

Section 1.1 Secure File Copy

Design

`secure_client` reads a local file, encrypts it with AES-256-CBC, and writes a binary frame to standard output. `secure_server` reads that frame from standard input, validates HMAC-SHA256 over the transmitted metadata and ciphertext, decrypts the payload, and atomically writes the output file only after authentication succeeds.

The shared key file is not transmitted. It is locally expanded into separate encryption and HMAC keys.

Frame properties:

- Magic: `NSC1`
- Length-prefixed output filename
- Length-prefixed ciphertext
- Random IV from OpenSSL `RAND_bytes`
- HMAC-SHA256 over header metadata, output filename, and ciphertext
- Binary-safe comparisons with OpenSSL constant-time comparison

Usage

Create a key:

```sh
make demo-key
```

Local test:

```sh
printf 'hello\n' > input.txt
./secure_client input.txt received.txt secure.key | ./secure_server secure.key .
```

Network transport with `ncat`:

```sh
ncat -l -p 5000 | ./secure_server secure.key .
./secure_client input.txt received.txt secure.key | ncat <server-ip> 5000
```

HMAC failure demonstration:

```sh
./secure_client input.txt received.txt secure.key -corrupt_data | ./secure_server secur
e.key .
```

Expected result: the server prints an HMAC validation failure and does not create the f
inal output file.

Section 1.2 Multi-User IRC/KDC Chat

Server

```sh
./server_real --bind 127.0.0.1 --kdc-port 9000 --chat-port 9001 --users users.db --root
 files
```

Options:

- `--bind HOST`: listener address.
- `--kdc-port PORT`: KDC/authentication port.
- `--chat-port PORT`: chat server port.
- `--users PATH`: user database.
- `--root DIR`: file root metadata value.

`users.db` format:

```text
alice:alicepass:files/alice
bob:bobpass:files/bob
carol:carolpass:files/carol
```

If `users.db` is absent, the server starts with demo users:

- `alice` / `alicepass`
- `bob` / `bobpass`
- `carol` / `carolpass`

Authentication Flow

The KDC port accepts an HMAC-authenticated `AUTH` request. User long-term keys are derived from the username and password with PBKDF2-HMAC-SHA256. On success, the server returns an AES-256-GCM protected response containing:

- a fresh session key,
- expiry time,
- and an encrypted chat-server ticket.

The chat port accepts the ticket and a session-key HMAC authenticator. Nonces are tracked server-side to reject replayed authenticators.

Client

```sh
./client_real --user alice --password alicepass --host 127.0.0.1 --kdc-port 9000 --chat-port 9001
```

Useful client options:

- `--file-root DIR`: directory used when this client sends files.
- `--download-dir DIR`: directory used for received TLS files.
- `--cert CERT`: self-signed certificate used when this client acts as TLS file receiver.
- `--key KEY`: private key for the TLS certificate.
- `--public-key TEXT`: public-key text advertised in the public-key workflow.

Supported Commands

```text
/who
/write all <message>
/create group <name>
/group invite <group-id> <user>
/group invite accept <group-id>
/request public key <user>
/send public key <user> <key>
/init group dhxchg <group-id>
/write group <group-id> <message>
/list user files <user>
/request file <user> <filename> <host> <port>
```

Group Messaging

Group creation, invitation, acceptance, public-key request/response routing, and group message delivery are implemented. `/init group dhxchg` initializes a group key for current group members and sends it only to group members over their authenticated chat sessions. This is a coursework demonstration of the required group-key workflow; it is not a production IRC or end-to-end encrypted messaging implementation.

File Listing And TLS File Transfer

`/list user files <user>` lists readable files from the selected user root. A requester can read another user's file only when an Assignment1-style sidecar ACL grants read permission.

`/request file <user> <filename> <host> <port>` starts the callback transfer:

1. Requester starts a short-lived TLS server on `<host>:<port>`.

2. Chat server verifies file access and notifies the owner client.
3. Owner client connects back over TLS and sends the file.
4. Requester saves the file into `--download-dir`.

Generate the requester certificate first:

```sh
make certs
```

Tests

Run all local tests:

```sh
tests/secure_copy_test.sh
tests/chat_smoke_test.sh
tests/group_demo.sh
tests/tls_file_test.sh
```

What the tests cover:

- successful secure-copy transfer,
- corrupt secure-copy HMAC rejection,
- KDC login success,
- bad-password rejection,
- `/who`,
- group creation/invite/accept/key/message workflow,
- ACL-authorized TLS file request and transfer.

Vulnerability Demonstrations

The test scripts demonstrate three defensive properties required by the rubric:

- Tampered ciphertext is rejected by HMAC validation in `tests/secure_copy_test.sh`.
- Invalid user password is rejected by KDC authentication in `tests/chat_smoke_test.sh`.
- Unauthorized file disclosure is prevented by ACL checks before file request routing; `tests/tls_file_test.sh` includes the positive authorized case and can be adapted by removing the ACL grant to show denial.

Known Boundaries

- The chat protocol is a coursework line protocol, not RFC IRC.
- User storage is a simple colon-delimited file for demonstration.
- Group key initialization is server-assisted and suitable for assignment demonstration only.
- Live multi-host deployment may require firewall and routing adjustments outside this repository.